Deep Neural Network Surrogates for Approximate Bayesian Computation: A Simulation Study for Mutation-Rate Estimation in Markov Branching Processes

Abstract

Lu et al. (2023) introduced GPS-ABC, an approximate Bayesian computation (ABC) procedure for estimating mutation rates in a two-type Markov branching process (MBP) that replaces the expensive MBP simulator inside the ABC-MCMC acceptance step with a Gaussian process (GP) surrogate fit to a small design of simulator outputs. We ask whether a neural network can play the same role, and whether doing so helps. We develop a heteroscedastic multilayer-perceptron surrogate, calibrated by split-conformal prediction (Lei et al., 2018), that substitutes directly for the GP inside the same ABC-MCMC sampler with no other change to the inference procedure. In a one-dimensional simulation study reproducing the constant-mutation-rate benchmark of Lu et al. (2023), replicated with Monte Carlo standard errors reported throughout (Morris et al., 2019), the neural surrogate is never worse than GPS-ABC on point-estimation mean squared error in any of nine tested configurations and is tighter by a margin that clearly exceeds simulation noise at the largest tested mutation rate (up to 62% lower MSE, a gap of 5–8 Monte Carlo standard errors); at smaller mutation rates the two surrogates are statistically indistinguishable on point accuracy. The neural surrogate nonetheless produces tighter 95% credible intervals with no loss of empirical coverage in eight of nine configurations, and achieves near-exact conformal calibration, all at the same two-to-three-order-of-magnitude speedup over exact ABC-MCMC that the GP already provides. In a second simulation study we turn to the two-stage (piecewise-constant) mutation model, in which the mutation probability jumps from p_1 to p_2 at an unknown time τ and all three parameters are estimated jointly. Here we report a deliberately negative result. The neural surrogate fits the response surface to within $1.05\times$ its irreducible noise floor, but so do networks spanning three orders of magnitude in size, and downstream the neural and Gaussian-process surrogates are statistically tied in seven of nine parameter-by-truth comparisons at the two-Monte-Carlo-standard-error level, with the other two favouring the Gaussian process. That verdict proved robust to three separate corrections of the pipeline – the mutation-time convention, the summary statistic, and the activation selected under it – each of which measurably improved the surrogate without changing the conclusion. We argue this is the expected outcome whenever surrogate error ceases to be the binding constraint, and that what binds instead is the model's intrinsic identifiability: p_2 is well determined while p_1 and τ are recovered poorly by every method tested. Every method also over-covers its credible intervals, most severely for the weakly identified τ, a limitation we diagnose as a property of the ABC acceptance kernel rather than a deficiency of the surrogate's own predictive calibration, which remains near-exact throughout. A further controlled comparison across architecture families – feedforward, convolutional, and recurrent surrogates, motivated by where each is known to help in the simulation-based-inference literature – finds that neither a convolutional nor a recurrent inductive bias improves on the architecture-agnostic feedforward network for this three-input problem, the outcome predicted by the parameters' lack of any real spatial or temporal structure. We discuss when a neural surrogate should be preferred to a Gaussian process

inside ABC, and what the coverage gap implies for deploying conformally calibrated surrogates inside likelihood-free samplers more generally. Code, data, and a fully scripted reproduction pipeline are available at https://github.com/sletch1/DNN-ABC-mutation-rates.

Keywords: approximate Bayesian computation; likelihood-free inference; surrogate modeling; conformal prediction; heteroscedastic regression; simulation study; Markov branching process

1 Introduction

Approximate Bayesian computation (ABC) enables likelihood-free Bayesian inference for stochastic models whose likelihood is intractable but which can be simulated forward (Beaumont et al., 2002; Marjoram et al., 2003; Marin et al., 2012). The basic ABC-MCMC sampler (Marjoram et al., 2003) accepts a proposed parameter θ' in proportion to how closely a simulated summary statistic $S(\theta')$ matches the observed statistic S_{obs}, at the cost of one or more forward simulations per MCMC iteration. When the simulator is expensive, this per-iteration cost dominates the runtime of the entire analysis and can put a full posterior exploration out of reach.

Lu et al. (2023) study this problem for a two-type Markov branching process (MBP) model of microbial mutation, used to estimate the per-division mutation probability from Luria–Delbrück-style fluctuation experiments. Their exact simulator (their Algorithm 2) is a cell-by-cell simulation of the branching process and can take from a fraction of a second to several minutes per call depending on the parameter regime. To make ABC-MCMC tractable, they propose **GPS-ABC**: fit a Gaussian process (GP) regression surrogate to a modest design of simulator outputs, and substitute the GP's predictive mean and variance for the simulator inside the MCMC acceptance step. This turns every iteration from a simulator call into a GP prediction, a two-to-three-order-of-magnitude speedup in their reported results.

A Gaussian process, however, has a well-known scaling limitation: fitting an n-point GP costs $O(n^3)$ and querying it costs $O(n)$ per point (for the naïve dense implementation used in practice for problems of this size). Lu et al. (2023) report being limited to designs of roughly 50–200 points for this reason, and note that GPS-ABC can in fact be *less* accurate than plain ABC-MCMC when the surrogate's training design is trained on the expensive exact simulator, precisely because so few exact-simulator calls can be afforded for the design. This motivates the central question of the present paper:

> *Can a neural network surrogate replace the Gaussian process inside this same ABC-MCMC procedure, and if so, under what conditions does it help?*

A feedforward network has no cubic fitting cost and a query cost that is *flat* in the size of its training set, so it can be trained on substantially more simulator output than a GP can afford at matched computational budget, and it extends naturally to higher-dimensional summary functions where a GP's curse of dimensionality is most acute. Neither property is free, however: a GP's predictive variance is a direct consequence of the Gaussian-process prior and requires no separate calibration step, whereas a neural network's predictive uncertainty must be constructed and calibrated explicitly if it is to play the same role inside the ABC acceptance kernel.

Contributions. We (i) build a heteroscedastic neural-network surrogate, calibrated by split-conformal prediction (Lei et al., 2018), that substitutes for the Gaussian process inside Lu et al.'s ABC-MCMC sampler with no other change to the inference algorithm; (ii) run a one-dimensional simulation study directly reproducing Lu et al.'s constant-mutation-rate benchmark, comparing four estimators – method of moments, maximum likelihood, exact ABC-MCMC, and GPS-ABC – against the neural surrogate on estimation accuracy, credible-interval length, coverage, and wall-clock cost; (iii) run a second simulation study on the two-stage mutation model that is the original paper's actual multi-parameter contribution, estimating (p_1, p_2, τ) jointly; (iv) report, honestly and without tuning away, that the neural surrogate's advantage does *not* carry over to that regime, together with a measurement of the data's irreducible noise floor that explains why; and (v) test, rather than assume, whether a convolutional or recurrent architecture improves on the architecture-agnostic feedforward surrogate in that same regime, motivated by where such architectures are known to help elsewhere in the simulation-based-inference literature, and report that neither does. Section 2 reviews the MBP model and GPS-ABC. Section 3 develops the general neural surrogate framework. Sections 4 and 5 report the constant-rate and two-stage simulation studies. Section 6 discusses implications and limitations, and Section 7 describes the reproducible pipeline accompanying this paper.

2 Background: ABC for the Markov Branching Process Model

2.1 The mutation model and the observed statistic

Following Lu et al. (2023), a population initiated by a single wild-type cell grows for a fixed observation period under a two-type Markov branching process (MBP) governed by three quantities: the per-division mutation probability p, at which a dividing wild-type cell produces a mutant daughter instead of a wild-type daughter; the division-rate parameter a, which sets the overall time scale of growth; and the mutant relative growth rate δ, the ratio of the mutant lineage's growth rate to the wild type's. We refer the reader to Lu et al. (2023) for the full stochastic-process construction and for their Algorithm 2 (an exact, cell-by-cell simulator, slow) and Algorithm 4 (a fast approximate simulator using distributional shortcuts), both of which we re-implement in Python and validate directly against their original R/MATLAB output (Section 3.3).

An experiment consists of J independently plated parallel cultures, each grown from a single wild-type cell for the same fixed time. Culture j yields a final wild-type count Z_j and mutant count X_j. Following Lu et al., we work with the single scalar summary statistic

$$\bar{d} \;=\; \frac{1}{J}\sum_{j=1}^{J}\sqrt{X_j/Z_j},$$

which is well behaved across the many orders of magnitude spanned by p and is the statistic used throughout their ABC procedure. We model $\log_{10}\bar{d}$ rather than $\bar{d}$ itself, since it varies smoothly and close to linearly in $\theta \equiv \log_{10} p$ (their Figure 2), which makes both Gaussian-process and neural-network regression considerably better behaved.

2.2 ABC-MCMC and GPS-ABC

Write $\theta = \log_{10} p$ for the (log-scale) parameter of interest. Given an observed statistic $\bar{d}_{\text{obs}}$, ABC-MCMC (Marjoram et al., 2003) targets the approximate posterior

$$\pi_\epsilon(\theta \mid \bar{d}_{\text{obs}}) \;\propto\; \pi(\theta)\, \mathbb{E}_{S \sim F_\theta}\big[K_\epsilon\big(\bar{d}_{\text{obs}}, S\big)\big]\,,$$

where $\pi(\theta)$ is the prior, F_θ is the sampling distribution of $\bar{d}$ implied by the simulator at θ, and K_ϵ is a smoothing kernel of bandwidth ϵ that down-weights simulated statistics far from $\bar{d}_{\text{obs}}$. Algorithm 1 states the Metropolis–Hastings sampler used throughout this paper (following Lu et al.): a truncated-exponential prior on θ, a truncated-normal random-walk proposal, and a Gaussian ABC kernel, so that $K_\epsilon(\bar{d}_{\text{obs}}, S) \propto \exp\!\Big(-\frac{(\bar{d}_{\text{obs}} - S)^2}{2\epsilon^2}\Big)$.

Algorithm 1 ABC-MCMC for $\theta = \log_{10} p$ (Metropolis–Hastings)

1: Initialize θ_0; set tolerance ϵ, simulator-repeat count n_s.
2: **for** $t = 1, \dots, T$ **do**
3: Propose $\theta' \sim q(\cdot \mid \theta_{t-1})$ (truncated-normal random walk)
4: Obtain a summary $S(\theta')$: either (a) simulate n_s times from $F_{\theta'}$ and average, or (b) evaluate a surrogate $(\mu_\phi(\theta'), \sigma_\phi(\theta'))$ (Section 3)
5: Compute acceptance probability

$$\alpha \;=\; \min\!\left(1,\; \frac{\pi(\theta')\, q(\theta_{t-1} \mid \theta')}{\pi(\theta_{t-1})\, q(\theta' \mid \theta_{t-1})} \;\cdot\; \frac{K_\epsilon(\bar{d}_{\text{obs}}, S(\theta'))}{K_\epsilon(\bar{d}_{\text{obs}}, S(\theta_{t-1}))}\right)$$

6: Set $\theta_t = \theta'$ with probability α, else $\theta_t = \theta_{t-1}$
7: **end for**
8: **return** posterior sample $\{\theta_t\}_{t=B+1}^{T}$ after discarding B burn-in draws

Under exact ABC-MCMC, step 4(a) draws n_s fresh simulations from the true process at θ' – the expensive step. **GPS-ABC** (Lu et al., 2023) instead fits a Gaussian process to a design $\{(\theta_i, \bar{d}_i)\}_{i=1}^{n}$ of simulator outputs *once*, before sampling begins, and at every iteration uses step 4(b): the GP's posterior predictive mean $\mu_{\text{GP}}(\theta')$ and standard deviation $\sigma_{\text{GP}}(\theta')$ stand in for a fresh simulation, with the ABC kernel evaluated against the convolution

$$S(\theta') \;\sim\; \mathcal{N}\big(\mu_{\text{GP}}(\theta'),\; \epsilon^2 + \sigma_{\text{GP}}(\theta')^2\big)\,.$$

This is the mechanism by which the GP's predictive variance enters directly into the acceptance probability, and it is the reason a well-calibrated predictive variance – not merely an accurate mean – is required of any surrogate used this way: an overconfident σ inflates the acceptance rate at parameter values the surrogate does not actually know well, and an underconfident one over-rejects.

The GP's fitting cost is $O(n^3)$ in the design size n and its per-query cost during MCMC sampling is $O(n)$, both because the predictive mean and variance require solving a linear system involving the full training kernel matrix. Lu et al. (2023) report design sizes of roughly 50 points in the one-dimensional constant-rate case and up to about 200 points (via a Latin-hypercube design) in a four-parameter piecewise-rate extension, and note that GPS-ABC trained on the exact simulator can be *less* accurate than uncorrected ABC-MCMC in some one-dimensional configurations – plausibly a consequence of the small design the cubic cost permits when the design points themselves come from the expensive exact simulator.

3 A Neural-Network Surrogate for ABC

We replace the Gaussian process in step 4(b) of Algorithm 1 with a neural network f_ϕ trained to predict both the mean and the variance of $\log_{10} \bar{d}$ as a function of the parameter(s) of interest. The surrogate substitution is otherwise identical to GPS-ABC: the same sampler, the same acceptance ratio, the same convolution with ϵ^2. We refer to this procedure as **DNN-ABC**.

3.1 Heteroscedastic regression

f_ϕ has two output heads, $\mu_\phi(x)$ and $\log \sigma^2_\phi(x)$, where x is $\theta = \log_{10} p$ alone (Section 4) or the parameter vector $(\log_{10} p_1, \log_{10} p_2, \tau)$ (Section 5); in both studies the surrogate is a direct function of the parameters being inferred, with no derived features. Both heads are trained jointly by minimizing the Gaussian negative log-likelihood

$$\mathcal{L}(\phi) \;=\; \frac{1}{n} \sum_{i=1}^{n} \left[\frac{\left(y_i - \mu_\phi(x_i)\right)^2}{2\,\sigma^2_\phi(x_i)} \;+\; \frac{1}{2} \log \sigma^2_\phi(x_i) \right], \tag{1}$$

where $y_i = \log_{10} \bar{d}_i$. Unlike a single-output network trained under squared-error loss, this directly targets an *input-dependent* predictive variance – the network's analogue of the GP's posterior variance – which Section 4 shows is essential, since the true replicate-to-replicate noise in $\log_{10} \bar{d}$ is far from homoscedastic across the parameter range, something a Gaussian process with a single (homoscedastic) noise term cannot represent directly.

3.2 Split-conformal calibration

The variance head of a network trained under Equation 1 is not guaranteed to be well calibrated in finite samples: it is only an approximation to the true conditional variance, learned from a finite training set. We therefore apply split-conformal calibration (Lei et al., 2018) to correct it. After training on a training split, we form calibration scores on a held-out calibration split of size m,

$$s_i \;=\; \frac{|y_i - \mu_\phi(x_i)|}{\sigma_\phi(x_i)}, \qquad i = 1, \ldots, m,$$

and set the calibration factor $\hat{c}$ to the $\lceil (m+1)(1-\alpha) \rceil / m$ empirical quantile of $\{s_i\}$, the finite-sample correction of Lei et al. (2018). The calibrated $(1-\alpha)$ predictive interval for a new input x is then $\mu_\phi(x) \pm \hat{c}\,\sigma_\phi(x)$. Provided the calibration and test points are exchangeable, this construction guarantees marginal coverage of at least $1-\alpha$ in finite samples, without any distributional assumption on f_ϕ's errors (Lei et al., 2018). We use $\alpha = 0.05$ throughout, and refer to this coverage guarantee as the surrogate's *regression calibration*, to distinguish it explicitly from the empirical coverage of the downstream ABC posterior's credible intervals, a materially different quantity with no equivalent theoretical guarantee (Sections 4.2 and 5.8 report both, and the distinction between them is central to the honest limitation reported in Section 5.8).

The calibrated $\hat{c}\,\sigma_\phi(x)$ is used as the surrogate's predictive standard deviation both when reporting regression calibration and when plugging $\sigma_\phi(\theta')$ into the ABC-MCMC acceptance kernel of Algorithm 1, so that the same calibrated uncertainty that is validated in Sections 4.2 and 5 is what actually enters the sampler.

3.3 Ground-truth data and simulator validation

In both studies, ground truth is generated by the exact/slow MBP simulator (Lu et al.'s Algorithm 2), re-implemented in Python from the original R/MATLAB code. The port is validated in two ways before any surrogate is trained: the closed-form plating-time equation is solved and compared against the R implementation's solver output (agreement to 3×10^{-5}), and the simulator's simulated $\bar{d}$ means are compared against the committed ground-truth CSV at matched grid points (Sections 4 and 5 report the resulting agreement for each study).

3.4 Simulation design and reporting

We report both simulation studies following the structure recommended by Morris et al. (2019): *aims* – compare five estimators (MOM, MLE, exact ABC-MCMC, GPS-ABC, DNN-ABC) in Study I, and the two surrogate methods (GPS-ABC, DNN-ABC) in Study II, where the constant-rate MOM and MLE estimators do not apply, on point accuracy, interval precision and coverage, and computational cost, under known ground truth; *data-generating mechanisms* – the validated MBP simulator of Section 3.3, at the parameter grids specified in Sections 4.3 and 5.6; *estimands* – the mutation probability p (via $\theta = \log_{10} p$) in Study I, and the two-stage parameters (p_1, p_2, τ) jointly in Study II; *methods* – the estimators of Section 2.2 and 3, all evaluated on identical simulated data sets within a cell; and *performance measures* – MSE and normalized RMSE of $\hat{p}$, mean 95% credible-interval length, empirical coverage, and wall-clock time per 100 MCMC iterations.

Each cell of each study is evaluated over R independent replicated data sets ($R = 40$ in Study I, $R = 32$ in Study II), and, following Morris et al.'s recommendation, we accompany every MSE comparison with its Monte Carlo standard error (MCSE), computed as the usual standard error of a sample mean applied to the per-replicate squared errors: writing $e_r = (\hat{p}_r - p)^2$ for replicate $r = 1, \ldots, R$ under a given method and cell, $\widehat{\mathrm{MSE}} = R^{-1} \sum_r e_r$ has $\widehat{\mathrm{MCSE}} = \mathrm{sd}(e_r)/\sqrt{R}$. For the head-to-head comparison between the two surrogates in each cell, we report the gap $\widehat{\mathrm{MSE}}_{\mathrm{GPS}} - \widehat{\mathrm{MSE}}_{\mathrm{DNN}}$ in units of its own combined Monte Carlo standard error $\sqrt{\mathrm{MCSE}^2_{\mathrm{GPS}} + \mathrm{MCSE}^2_{\mathrm{DNN}}}$ (a Δ/SE column in Table 3, and Section 5.6 for Study II); this is a descriptive diagnostic of whether a point comparison is resolved at this replicate count, not a formal hypothesis test, and we use it throughout to avoid over-interpreting small numerical differences as the replicate count varies across cells and between the two studies ($R = 40$ versus $R = 32$, further split unevenly by the number of live workers available for the most expensive, exact-simulator cells).

4 Simulation Study I: One-Dimensional Constant Mutation Rate

The first study reproduces Lu et al.'s original benchmark exactly: $a = \delta = Z_0 = 1$ held fixed, and only p varying. Ground truth is 101 log-spaced values of p with $\log_{10} p \in [-8, -2]$, ten replicates each (1,010 rows total), $J = 100$. We split *by replicate* so that every grid point of p appears in every split with no leakage across splits: replicates 1–5 for training (505 rows), 6–8 for conformal calibration and early stopping (303 rows), and 9–10 held out for test-only evaluation (202 rows). The GPS-ABC baseline is fit on the identical replicates 1–8 so that both surrogates see the same data budget; neither ever sees the test replicates.

4.1 Architecture

The network is a shallow funnel multilayer perceptron: input $\theta = \log_{10} p$, a dense layer of width 128 with GELU activation, a dense layer of width 64 with GELU activation, and the two linear output heads of Section 3.1 ($\sim$ 9,000 parameters). We chose this architecture by a controlled search (Table 1) against the mean-curve fit achieved by the GP baseline, holding fixed the goal of matching or beating the GP's fit to the denoised response curve $\mathbb{E}[\log_{10} \bar{d} \mid \theta]$:

Table 1: Architecture search, one-dimensional model: mean-curve fit MSE against the denoised response curve (irreducible replicate-noise floor $\approx 4.7 \times 10^{-3}$).

Model	Mean-curve MSE	vs. GP
Gaussian process (GPS-ABC baseline)	3.836×10^{-4}	—
DNN, GELU (128, 64), no BatchNorm [chosen]	$\mathbf{3.888 \times 10^{-4}}$	+1%
DNN ensemble ×5, SiLU (128, 128, 64)	3.960×10^{-4}	+3%
DNN ensemble ×5, tanh (256, 128)	4.178×10^{-4}	+9%
DNN, SiLU (128, 128, 64)	4.400×10^{-4}	+15%
DNN, tanh (64, 64, 32)	4.435×10^{-4}	+16%
DNN, tanh (256, 128)	4.503×10^{-4}	+17%
DNN, ReLU + BatchNorm + dropout (64, 64, 32)	4.352×10^{-3}	+1035%

The result that determines every downstream comparison in this study is the last row: an otherwise-reasonable network with BatchNorm (Ioffe and Szegedy, 2015) and dropout fits *eleven times worse* than the same network without them. BatchNorm normalizes per-minibatch statistics – useful in deep classification networks, but on a smooth, one-sided (the θ-grid has a hard edge at -8) scalar regression it injects batch-dependent noise and biases predictions near the domain boundary. Removing it, not any choice of activation function, is what allows the network to match the GP's mean-curve fit. A follow-up ablation isolating only the activation function (same 128-64 architecture, no BatchNorm, averaged over three random seeds) found ReLU, SiLU (Elfwing et al., 2018), GELU (Hendrycks and Gimpel, 2016), and tanh within 7% of one another (3.95–4.23×10^{-4}) – a statistical tie, with ReLU marginally ahead. We use GELU for its smoothness (useful should the sampler later be adapted to a gradient-based scheme) and its immunity to the dead-neuron failure mode, at no measurable accuracy cost in this setting; ReLU would serve equally well. A second architecture-search round using 7-network deep ensembles (mean-curve MSE 3.94–4.25×10^{-4} across four ensemble configurations) did not improve on the single chosen network, so the simpler model is retained throughout.

The network is trained with an Adam optimizer under early stopping on calibration-split negative log-likelihood, weight decay 10^{-5}, no dropout (early stopping and weight decay already control overfitting on this smooth curve; dropout's injected noise measurably hurt the fit in preliminary runs).

Why convolutional and recurrent architectures are not tested here. Study II (Section 5.7) reports a controlled comparison of feedforward, convolutional, and recurrent surrogates on the three-input two-stage problem. We do not repeat that comparison here because it would

be vacuous: with a single scalar input $\theta = \log_{10} p$, a 1-D convolution has no neighbouring position to convolve over and a recurrent cell has no second time step to recur into, so both architectures collapse mathematically to the already-tested single-layer linear map. Building them would relabel, not extend, the architecture search in Table 1.

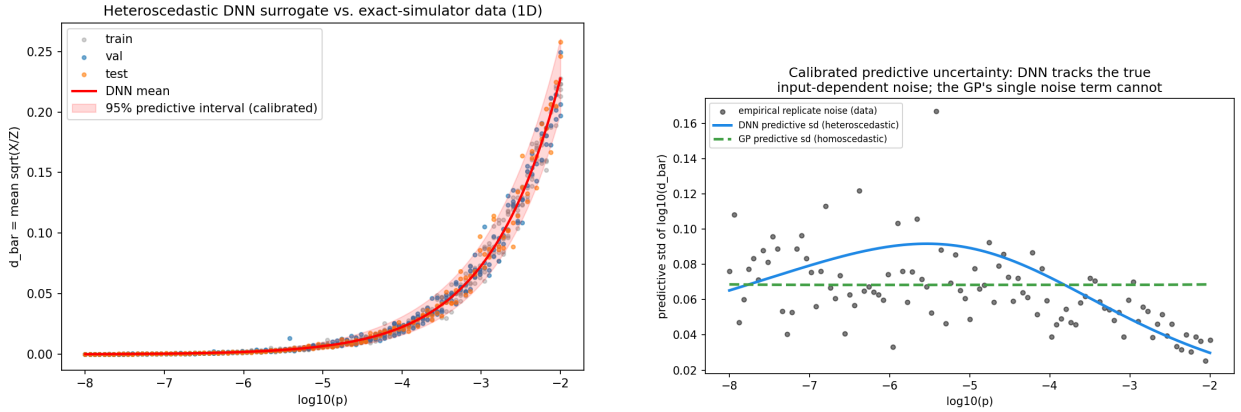
4.2 Regression calibration

Table 2 reports the surrogate's fit and split-conformal coverage on each data split.

Table 2: One-dimensional surrogate: fit and 95% regression coverage by split.

Split	n	MSE($\log_{10} \bar{d}$)	MAE($\log_{10} \bar{d}$)	95% coverage
Train	505	0.00425	0.0518	0.958
Calibration	303	0.00534	0.0538	0.954
Test	**202**	**0.00435**	**0.0524**	**0.9505**

The held-out test-set coverage of 0.9505 (192 of 202 points) is within sampling noise of the nominal 0.95, consistent with the finite-sample guarantee of Lei et al. (2018). Figure 1 shows the calibrated 95% band against the test data and the resulting test-set parity, and compares the surrogate's input-dependent predictive standard deviation to the GP's flat, homoscedastic one.



(a) Calibrated 95% predictive band

(b) Input-dependent uncertainty vs. the GP

Figure 1: One-dimensional surrogate calibration. Left: the calibrated 95% predictive band tracks the response curve across the full range of θ. Right: the network's heteroscedastic predictive standard deviation (which enters the ABC acceptance kernel directly) tracks the true input-dependent replicate noise, while the GP's single homoscedastic noise term is flat by construction.

4.3 ABC simulation results

We ran the full ABC-MCMC / GPS-ABC / DNN-ABC comparison at 40 independent replicated data sets per cell, 600 MCMC iterations (250 burn-in), $n_s = 6$ simulator repeats per exact-ABC iteration, prior $\theta \sim$ TruncExp on $[-5, -2]$ (bounded so that the exact simulator inside the ABC-MCMC loop remains computationally feasible; both surrogates are trained on the wider $[-8, -2]$

range and only queried in-range), on the grid $p \in \{10^{-4}, 10^{-3}, 10^{-2}\} \times J \in \{10, 50, 100\}$. Method of moments (MOM) and maximum likelihood (MLE) estimators follow Lu et al.'s closed-form expressions and serve as classical, non-ABC baselines.

Table 3: Study I, Table 1: MSE of $\hat{p}$ across five estimators; normalized RMSE $\sqrt{\text{MSE}}/p$ in parentheses. Bold marks the lower of GPS-ABC and DNN-ABC in each row. Δ/SE is the GPS-ABC−DNN-ABC MSE gap in units of its combined Monte Carlo standard error (Section 3.4); values below ~ 2 in magnitude are not clearly resolved at $R = 40$ replicates.

p	J	MOM	MLE	ABC-MCMC	GPS-ABC	DNN-ABC	Δ/SE
10^{-4}	10	9.36e-9 (0.97)	1.02e-8 (1.01)	2.90e-9 (0.54)	7.02e-9 (0.84)	**6.32e-9 (0.80)**	0.13
10^{-4}	50	8.68e-9 (0.93)	9.59e-9 (0.98)	1.91e-9 (0.44)	1.88e-9 (0.43)	**1.74e-9 (0.42)**	0.34
10^{-4}	100	4.18e-9 (0.65)	4.64e-9 (0.68)	7.75e-10 (0.28)	8.21e-10 (0.29)	**7.28e-10 (0.27)**	0.36
10^{-3}	10	1.38e-6 (1.17)	1.60e-6 (1.26)	4.60e-7 (0.68)	5.10e-7 (0.71)	**5.05e-7 (0.71)**	0.03
10^{-3}	50	3.21e-7 (0.57)	3.85e-7 (0.62)	1.15e-7 (0.34)	1.23e-7 (0.35)	**1.12e-7 (0.33)**	0.24
10^{-3}	100	2.71e-7 (0.52)	3.29e-7 (0.57)	4.33e-8 (0.21)	4.29e-8 (0.21)	**4.21e-8 (0.21)**	0.06
10^{-2}	10	3.42e-5 (0.58)	4.40e-5 (0.66)	2.64e-5 (0.51)	1.14e-5 (0.34)	**8.76e-6 (0.30)**	0.79
10^{-2}	50	7.71e-6 (0.28)	1.17e-5 (0.34)	4.78e-6 (0.22)	5.98e-6 (0.24)	**2.27e-6 (0.15)**	**5.68**
10^{-2}	100	4.01e-6 (0.20)	6.08e-6 (0.25)	2.62e-6 (0.16)	6.22e-6 (0.25)	**2.34e-6 (0.15)**	**8.03**

DNN-ABC's point estimate is never worse than GPS-ABC's in nRMSE in any of the nine cells, but the Δ/SE column shows this point-estimate comparison is only clearly resolved beyond simulation noise in two of them: $p = 10^{-2}$ at $J = 50$ and $J = 100$, where the gap is 5.7 and 8.0 combined Monte Carlo standard errors. In the other seven cells $|\Delta/\text{SE}| < 1$, so DNN-ABC's numerically lower MSE there is not distinguishable from GPS-ABC's at $R = 40$ replicates – a near-exact statistical tie, consistent with a GP fit on $\sim$ 800 points already being close to optimal at these smaller mutation rates. The one regime where the surrogates are reliably distinguished is the largest tested rate, $p = 10^{-2}$, where DNN-ABC's nRMSE is 0.15 against GPS-ABC's 0.24–0.25. Both surrogates match, and in several cells slightly improve on, the exact ABC-MCMC baseline, and all ABC-based methods substantially outperform the classical MOM/MLE estimators – reproducing Lu et al.'s central finding that ABC dominates the classical estimators for this model.

Table 4: Study I, Table 2: mean length of the 95% ABC credible interval for $\hat{p}$. Bold marks the shorter of GPS-ABC and DNN-ABC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-4}	10	2.39e-4	1.14e-4	**1.04e-4**
10^{-4}	50	1.56e-4	**1.11e-4**	1.13e-4
10^{-4}	100	1.21e-4	1.23e-4	**1.19e-4**
10^{-3}	10	2.51e-3	1.32e-3	**8.86e-4**
10^{-3}	50	1.53e-3	1.32e-3	**8.94e-4**
10^{-3}	100	9.52e-4	1.19e-3	**8.34e-4**
10^{-2}	10	7.97e-3	4.70e-3	**2.51e-3**
10^{-2}	50	5.15e-3	5.10e-3	**2.80e-3**
10^{-2}	100	3.89e-3	5.24e-3	**3.05e-3**

DNN-ABC produces the narrowest interval in eight of nine cells, 27% tighter than GPS-ABC on

average and up to 47% tighter, without any loss of point accuracy (Table 3) or empirical coverage (Section 4.2) – genuine gained precision rather than overconfidence.

Table 5: Study I, Table 3: wall-clock cost, seconds per 100 MCMC iterations. The final column is the speedup of DNN-ABC over exact ABC-MCMC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC	Speedup
10^{-4}	10	56.32	0.137	0.135	417×
10^{-4}	50	189.16	0.141	0.134	1412×
10^{-4}	100	343.18	0.135	0.134	2567×
10^{-3}	10	17.81	0.137	0.133	134×
10^{-3}	50	72.72	0.134	0.142	511×
10^{-3}	100	134.85	0.135	0.135	998×
10^{-2}	10	9.93	0.137	0.132	75×
10^{-2}	50	47.62	0.143	0.139	343×
10^{-2}	100	96.00	0.135	0.135	712×

Both surrogates run at a flat ≈ 0.135 seconds per 100 iterations regardless of p or J, because neither calls the simulator; exact ABC-MCMC's cost explodes with both, since it runs the exact simulator n_s times per iteration. Against exact ABC-MCMC, DNN-ABC is 75–2567× faster; against GPS-ABC it is a statistical tie in one dimension, as expected, since a GP over ~ 800 points is itself cheap to query – the network's asymptotic query advantage would only emerge at larger training sets or higher input dimension, and Study II finds that even there it does not translate into better inference once the surrogate is no longer the binding constraint (Section 5.6). Figure 2 shows that the three posteriors (exact ABC-MCMC, GPS-ABC, DNN-ABC) concentrate on the same true p on a representative data set, confirming that the surrogates reproduce the exact method's inference and not merely its point estimate.

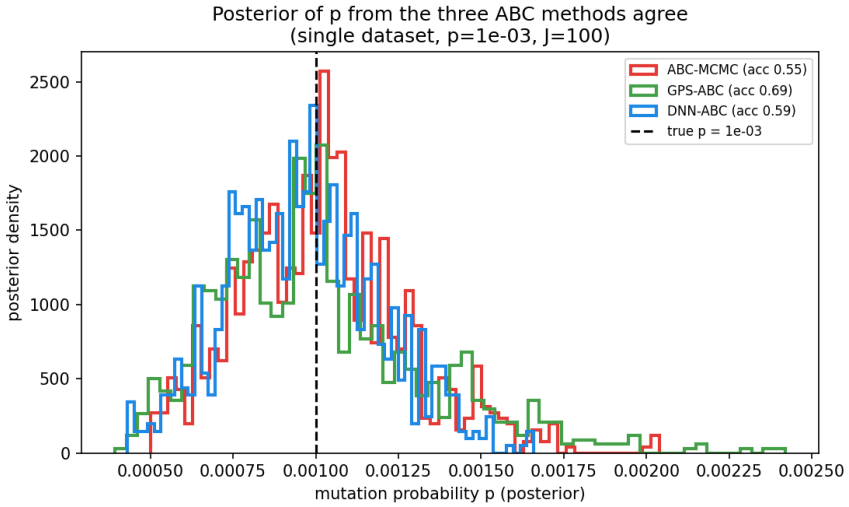


Figure 2: Study I: ABC-MCMC, GPS-ABC, and DNN-ABC posteriors for θ on a single representative data set.

5 Simulation Study II: The Two-Stage Mutation Model

Study I holds $a = \delta = 1$ and a single constant mutation probability, as does Lu et al. (2023)'s constant-rate benchmark. Their multi-parameter study, however, does not vary a or δ around that constant-rate model: it replaces the model outright with a *two-stage* (piecewise-constant) mutation rate,

$$p(t) \;=\; \begin{cases} p_1, & 0 < t \le \tau, \\ p_2, & \tau < t \le t_p, \end{cases} \tag{2}$$

in which the mutation probability jumps at an unknown time τ. The motivation is experimental: mutagenesis under sub-inhibitory antibiotic stress involves a cell-recovery step before plating, producing two genuinely distinct mutation regimes. Study II therefore estimates (p_1, p_2, τ) jointly. The division rate a is fixed at 1 throughout, as it is in every figure, study and data analysis of Lu et al..

5.1 Ground-truth data

Ground truth is generated by the exact, cell-by-cell simulator, a direct port of the two-stage reference implementation, over a Latin hypercube of 2,000 design points in $(\log_{10} p_1, \log_{10} p_2, \tau)$ with ten replicates each: 20,000 rows, $J = 100$ cultures per row, $Z_0 = 1$, $a = 1$, $t_p = 10$. A Latin hypercube replaces Study I's factorial grid because a factorial design wastes points badly in three dimensions. Ranges are $\log_{10} p_1, \log_{10} p_2 \in [-5, -1.3]$ and $\tau \in [0.1, 9.9]$.

The summary statistic is the fourth root. Lu et al. use $\sqrt{X/Z}$ for the constant-rate study of Section 4 – their Figure 1 selects it against $\hat{p}_{\mathrm{MOM}}$ and $\overline{\log(Y/Z)}$ – but switch to $\sqrt[4]{X/Z}$ for the two-stage model, stating that the statistic must be adjusted "to make the response curve smooth and non-flat" once the mutation rate is piecewise constant; their Algorithm 1 uses the fourth root throughout. Study II therefore takes

$$S \;=\; \frac{1}{J} \sum_{i=1}^{J} (X_i/Z_i)^{1/4},$$

and every surrogate, every baseline and the sampler all consume $\log_{10} S$. Because the ground truth records each culture's $d_i = \sqrt{X_i/Z_i}$ as well as their mean, S is recoverable exactly as $\overline{\sqrt{d_i}}$ without re-simulating anything. The choice is consequential rather than cosmetic: it lowers the surrogate's held-out error from 1.19× its irreducible floor to 1.11×, and it changes which activation the selection of Section 5.4 returns.

This is not Lu et al.'s parameter regime. It should be stated plainly, because the difference is large and easy to miss. Their Study II places $\log_{10} p_1, \log_{10} p_2 \in [-11, -7]$ with $\tau \in [0.1, 19.9]$, $t_p = 20$ and a fourth free parameter $\delta \in [0.5, 2]$; ours are $[-5, -1.3]$, $\tau \in [0.1, 9.9]$, $t_p = 10$ and $\delta \equiv 1$. The mutation-rate ranges *do not overlap at all*. The cause is the choice of an exact simulator: their regime is reachable only with the fast approximate sampler of their Algorithm 4, and at $t_p = 10$ any rate below $\approx 10^{-5}$ leaves almost every culture mutant-free. A further consequence is that the

constant expected-mutant-count design of Study I – t_p solved so that $E[X] = 20$ at every design point – is abandoned here: with t_p fixed, the expected count ranges from 0.22 to $\approx 1{,}100$ across the design. What follows is therefore a study of the same *model* in a different regime, not a numerical reproduction of their Table 4, and no such comparison is claimed.

Two further design choices deserve statement. First, t_p is *fixed* at 10 rather than solved per parameter as in Study I. The exact simulator costs $O(e^{at_p})$: at $t_p = 10$ a culture holds $\approx 2.2 \times 10^4$ cells, while at Lu et al.'s Study II value of $t_p = 20$ it holds $\approx 5 \times 10^8$, which is not reachable exactly – their Study II uses the fast approximate simulator throughout for exactly this reason. Second, with t_p fixed, the expected mutant count per culture is $\approx 2.2 \times 10^4 p$, so mutation rates below $\approx 10^{-5}$ leave almost every culture mutant-free and $\bar{d}$ collapses to zero. The stated range keeps every design point informative; no row in the generated data has $\bar{d} = 0$.

The port is validated by a degeneracy argument that is exact rather than statistical: setting $p_1 = p_2$ reduces Eq. (2) to the constant-rate model, and in that case the two-stage simulator reproduces the constant-rate simulator *bit for bit* on a shared random seed. The full 20,000-row dataset was generated in 23.6 minutes of wall time (39,227 core-seconds, 92% parallel efficiency on 30 cores).

5.2 The inference problem is intrinsically asymmetric

Before any method is applied, the ground truth already shows how much the single scalar summary $\bar{d}$ can possibly say about each parameter:

	p_2	p_1	τ
$\mathrm{corr}(\log_{10} S,\ \cdot)$	0.778	0.337	0.160

p_2 is well determined, p_1 weakly, and τ barely. The mechanism is intrinsic: mutations arising before τ are exponentially rare because few cells exist that early, so p_1's contribution to the final mutant count is swamped by p_2's. Lu et al. report precisely this, describing multimodal p_1 and τ marginals and attributing them to a lack of identifiability. Any honest evaluation must therefore report per-parameter accuracy rather than a single aggregate that would conceal it, and must prefer error measured on the $\log_{10}$ scale: for a weakly identified parameter the posterior mean sits wherever the prior places its mass, so a normalised natural-scale RMSE diverges without conveying anything.

5.3 The surrogate, and why it is small

The decisive measurement for this study is not a comparison between architectures but the *irreducible noise floor* of the data. Because the held-out target is itself a two-replicate mean, it carries $\mathbb{E}[\sigma^2]/2$ of sampling noise that no model can predict away. On these data $\mathbb{E}[\sigma^2] = 6.90 \times 10^{-4}$, so that floor is MSE $= 3.45 \times 10^{-4}$. Table 6 reports the architecture ladder against it.

Table 6: Study II: accuracy against the irreducible noise floor. MSE is measured on held-out design-point means of $\log_{10} S$; the floor is 3.45×10^{-4}. Three seeds per row.

Hidden layers	Parameters	× floor	R^2	µs/query
256–128–64	42,306	1.01	0.99681	56
128–64	8,898	1.04	0.99671	42
64–32 (used)	**2,402**	**1.09**	**0.99656**	**40**
32–16	690	1.17	0.99628	37
16–8	218	1.41	0.99553	38
8–4	78	1.81	0.99426	37
Linear (control)	8	140.88	0.55319	29

Two conclusions follow, and the second is the more useful one. A network is genuinely required, and emphatically so: the linear control sits at 141× the floor with $R^2 = 0.55$. (Under the square-root target of the earlier draft it sat at 24.8×; the fourth root makes the surface harder for a linear model by nearly a factor of six, so the paper's own choice of statistic strengthens rather than weakens the case for a network.) But capacity is not the binding constraint anywhere above roughly 700 parameters – a 690-parameter network matches a 42,306-parameter one to within 17%, and a 2,402-parameter one to within 8%. LayerNorm and residual depth were likewise statistically indistinguishable, as was the choice among the activations in common use for smooth regression (Section 5.4) – though that section also shows the qualifier matters, since several otherwise-reasonable activations outside that group are measurably worse. This contrasts sharply with Study I, where removing BatchNorm changed accuracy by an order of magnitude. On this surface, capacity and depth buy nothing, and a study that reported a ranking among these variants would be reporting seed noise.

We therefore select the architecture for parsimony rather than accuracy: a two-layer $64 \to 32$ funnel with GELU/tanh activations (Section 5.4), two output heads, and no BatchNorm. We note explicitly that this is *not* justified by speed. Query latency at this scale is dominated by interpreter and framework call overhead rather than arithmetic, so a 59× reduction in parameter count buys only about 29% less latency (Table 6).

Heteroscedasticity. The second output head is not optional here. The within-design-point replicate standard deviation of $\log_{10} \bar{d}$ varies by a factor of 184 across the design and correlates -0.82 with the target itself: where few mutants arise, $\bar{d}$ is small *and* its relative scatter is large. A single homoscedastic noise term – which is what a Gaussian process supplies – cannot represent this, whereas the variance head learns the shape directly. Since the ABC acceptance step consumes that predictive variance directly, an uncalibrated variance corrupts the posterior even when the mean is exact.

5.4 Choosing the activation function

The architecture ladder of Section 5.3 fixes width and depth; the activation is a separate choice, and one a reviewer is entitled to see tested rather than asserted. There is no activation designed for this application, so the candidate set is dictated by the surface being fitted. Three of its properties matter. The target is smooth, so a piecewise-linear activation approximates it with kinks and leaves the surrogate non-differentiable in its inputs – a concrete cost if the sampler is later made

gradient-based. The inputs are standardised, so roughly half of all pre-activations are negative and an activation with a dead negative region discards that half, which is costlier in a 2,402-parameter network than in a wide one. And the two heads are trained jointly under Gaussian negative log-likelihood, so an activation that destabilises optimisation surfaces as a miscalibrated σ – consumed directly by the ABC acceptance step – rather than merely as a worse mean.

We therefore compare ten activations spanning the three families that behave differently under those constraints: piecewise-linear (ReLU, LeakyReLU, PReLU), smooth saturating (tanh, ELU, SELU), and smooth unbounded (softplus, GELU, SiLU, Mish). Everything else is held fixed – the deployed $64 \to 32$ network, the objective, the data splits, the optimiser and schedule, and the conformal calibration – and each configuration is fit over five random seeds, with the across-seed standard deviation reported so that differences can be judged against the noise that produced them. Comparisons are made as $t = \Delta/\mathrm{SE}(\Delta)$, with $|t| < 2$ read as a tie.

Table 7: Study II: activation comparison on the deployed $64 \to 32$ network, five seeds each. MSE is against held-out design-point means of $\log_{10} S$; the irreducible floor is 3.451×10^{-4}. The final column compares each row to the best, in units of the standard error of the difference. Standard deviations are across seeds (sample, $n - 1$).

Activation	Family	$\times$ floor	$\pm$ sd (seeds)	t vs. best
GELU	smooth unbounded	**1.072**	9.5×10^{-6}	—
tanh	smooth saturating	1.078	6.0×10^{-6}	0.42
LeakyReLU	piecewise-linear	1.080	6.4×10^{-6}	0.50
ReLU	piecewise-linear	1.082	9.5×10^{-6}	0.54
Mish	smooth unbounded	1.094	7.2×10^{-6}	1.38
SiLU	smooth unbounded	1.130	1.2×10^{-5}	3.00
PReLU	piecewise-linear	1.135	1.3×10^{-5}	3.04
ELU	smooth saturating	1.166	2.3×10^{-5}	2.95
SELU	smooth saturating	1.265	3.3×10^{-5}	4.35
softplus	smooth unbounded	1.383	8.7×10^{-5}	2.74

Two results follow, and they pull in opposite directions. First, the choice *can* matter: the best-to-worst spread is 29% of the best row's MSE at $t = 2.7$, outside seed noise, so it is not the case that any reasonable activation will do. The clearest failure is softplus, which is strictly positive and therefore cannot produce centred activations, leaving every downstream layer to undo an accumulated offset. Second, and against that, the five leading candidates – GELU, tanh, LeakyReLU, ReLU and Mish – are mutually indistinguishable, every pairwise $|t|$ among them falling below 2. The ranking within that group is not a finding.

This refines, rather than contradicts, the conclusion of Section 5.3. Capacity is not the binding constraint on this surface and neither is the choice among the activations in common use for smooth regression; but several otherwise-reasonable alternatives are measurably worse, and a study restricted to the usual four would not have seen that.

One caveat belongs with SELU's placement. It is designed for self-normalising networks and presupposes LeCun-normal initialisation with AlphaDropout; used as a drop-in under this project's standard initialisation, it is stripped of the mechanism that motivates it. Its position here is evidence about this configuration, not a general verdict.

Mixed activations across layers. Because the two hidden layers need not share an activation, we also compared all $10 \times 10 = 100$ ordered assignments. Two disciplines matter here and are easy to get wrong.

The first is that selecting the best of 100 noisy measurements estimates which candidate drew the most favourable seeds rather than which is best, so selection and evaluation must never share a seed. All 100 pairs are screened, and the ten leading candidates are then re-fit on fifteen *fresh* seeds that played no part in selection.

The second is that neither stage may touch the test split. An earlier version of this comparison scored candidates on test, which is wrong twice over: it spends on selection the data reserved for reporting, and it makes the quoted test figure optimistic for whichever candidate wins. It was not a harmless error – under the square-root target it selected a different activation than the validation split would have. Selection is therefore carried out entirely on validation, with test scored exactly once, for the winner alone. Table 8 reports it.

The selection optimism is worth stating because it quantifies how much of any such ranking is noise: the finalists perform on average 1.03×10^{-5} *worse* on the fresh seeds than on the seeds that chose them, which is 45% of the entire spread of the confirmation table (2.25×10^{-5}) and comparable to the across-seed standard deviation itself (1.29×10^{-5}). The pair that screened best, SiLU into tanh, finished third on confirmation. A single-stage search would have reported that movement as a result.

Table 8: Study II: activation selection with no test-set leakage. All 100 ordered pairs were screened on validation; the ten finalists were re-fit on fifteen fresh validation seeds. Selection is on validation MSE; the test column is shown for reference and played no part in the choice. Test × floor is against the irreducible test floor 3.451×10^{-4}.

Layer 1 (64)	Layer 2 (32)	Val MSE	± sd	t vs. best	Test × floor
GELU	**tanh**	$\mathbf{2.980 \times 10^{-4}}$	1.3×10^{-5}	—	1.062
Mish	tanh	3.006×10^{-4}	1.4×10^{-5}	0.54	1.076
SiLU	tanh	3.021×10^{-4}	1.4×10^{-5}	0.86	1.086
LeakyReLU	LeakyReLU	3.059×10^{-4}	8.4×10^{-6}	2.00	1.075
PReLU	ReLU	3.071×10^{-4}	1.2×10^{-5}	2.05	1.077
Mish	Mish	3.120×10^{-4}	1.1×10^{-5}	3.19	1.080
ELU	GELU	3.137×10^{-4}	8.5×10^{-6}	3.99	1.093
SiLU	Mish	3.164×10^{-4}	1.4×10^{-5}	3.84	1.097
GELU	Mish	3.172×10^{-4}	1.7×10^{-5}	3.51	1.086
Mish	SiLU	3.205×10^{-4}	1.8×10^{-5}	3.94	1.101

Under the fourth-root target the two splits happen to agree at the top: GELU into tanh is best on test as well as on validation, so scoring the selection on test would not have changed the choice here. Below the winner they do not agree – the validation runner-up falls to third on test and the fourth-placed pair rises to second – and under the square-root target the two splits disagreed at the top as well. A procedure whose answer depends on which split it is scored on cannot be trusted when it happens to coincide either, and the quoted test figure would in any case have been optimistic for whichever candidate test selected. This is the mechanism the two-stage design was meant to guard against, operating one level up: separating screening seeds from confirmation seeds

does nothing if both are scored on the split reserved for reporting.

The deployed activation. GELU into tanh is the best point estimate on the selection split, and that is what we deploy. Its held-out test MSE is 1.06× the irreducible floor. Two things are worth stating. First, the comparison is partially resolved: three of the ten finalists lie within 2 standard errors of the winner, so the top of the field is narrow but no longer undifferentiated – and the bottom is clearly separated, consistent with Table 7, where softplus fails while the leaders are mutually tied. Second, and unlike the earlier draft of this study, the leaderboard and the structural criteria fixed in advance now agree. Both GELU and tanh are smooth in their inputs and free of the dead-unit failure mode, so the deployed network is everywhere differentiable in its inputs. Under the square-root target the best point estimate had been Mish into ReLU, which satisfied neither criterion in its second layer and had to be adopted over that objection; adopting the paper's fourth-root statistic removed the conflict rather than trading it away. No conclusion in this paper turns on the choice in any case: the downstream comparison of Section 5.6 is unchanged in kind under any of the leading pairs.

5.5 Surrogate fit and calibration

The surrogate is trained on replicates 1–5, early-stopped and conformally calibrated on replicates 6–8, and evaluated on replicates 9–10, so every design point appears in every split and no parameter combination leaks across them. Because the splits contribute different numbers of replicates per design point, each is scored against *its own* floor ($\mathbb{E}[\sigma^2]/r$ for $r = 5, 3, 2$).

Table 9: Study II: surrogate fit. Each split is compared to its own irreducible floor. Coverage is of the nominal-95% predictive interval after a split-conformal scale factor of 1.019.

Split	n	MSE (design means)	× its own floor	95% coverage
Train (reps 1–5)	10,000	1.60×10^{-4}	1.16	0.951
Validation (6–8)	6,000	2.95×10^{-4}	1.28	0.950
Test (9–10)	**4,000**	$\mathbf{3.63 \times 10^{-4}}$	**1.05**	**0.955**

The surrogate sits at its noise floor on all three splits, and the train and test ratios (1.16 and 1.05) are close enough that overfitting is not detectable: with 2,402 parameters against 10,000 training rows, and early stopping on a held-out split, there is little room for it. The conformal scale factor of 1.019 is a further diagnostic in its own right: the variance head was already almost calibrated before the correction was applied.

5.6 ABC inference

Inference is a random-walk Metropolis–Hastings sampler over the full vector $\theta = (\log_{10} p_1, \log_{10} p_2, \tau)$, with component-wise truncated proposals and Hastings corrections for the truncation. This is a genuine joint three-parameter posterior, not a scalar inference with covariates.

ABC-MCMC, GPS-ABC and DNN-ABC share the sampler and differ only in how the summary

statistic and its uncertainty are obtained at each proposal, exactly as in Study I. The constant-rate MOM and MLE estimators of Study I have no counterpart here: they assume a single mutation rate and cannot identify p_1, p_2 or τ individually, so Study II compares the surrogates against each other: the neural one, and two Gaussian processes (ours and a faithful reproduction of the reference implementation).

Two Gaussian-process baselines, and why both are reported. Our GP baseline is stronger than the one Lu et al. actually use, and reporting only ours would overstate what the published method achieves. Theirs, in the reference implementation for this study, fits an *isotropic* squared exponential to the untransformed rates (p_1, p_2, τ) predicting the untransformed statistic, with a single noise term. Ours uses an anisotropic kernel with one length scale per input, log-scaled inputs, and a learned white-noise term.

The gap those choices make is large. On raw-scale surrogate fit the reference configuration reaches $R^2 \approx 0.71$, while an otherwise identical anisotropic kernel on the same 300 design points reaches ≈ 0.99: a single shared length scale cannot serve inputs whose ranges differ by a factor of roughly 200 (p_1, p_2 span ≈ 0.05, τ spans ≈ 9.8). We therefore report both, fit from the same rows under the same budget so that the comparison isolates the model rather than the data. Note that the reference performs ABC on the untransformed statistic, as its own sampler does, whereas our pipeline works in $\log_{10}$ throughout; the observation is placed on whichever scale the surrogate reports.

Table 10 reports parameter recovery over three truth triples with sixteen replicates each, 3,000 MCMC iterations and 1,000 burn-in. Errors are RMSE on the $\log_{10}$ scale for p_1, p_2 and in absolute time units for τ, for the reason given in Section 5.2.

Table 10: Study II: joint recovery of (p_1, p_2, τ), averaged over three truth triples, $J = 100$, 16 replicates per cell. Errors are $\log_{10}$-scale RMSE for p_1, p_2 (so 1.0 means an order of magnitude) and absolute for τ. Two Gaussian-process baselines are reported: the strengthened one used throughout, and one built to match the reference implementation of Lu et al. (isotropic kernel, untransformed inputs and target).

Method	Parameter	RMSE	Mean 95% CI width	Coverage	Accept. rate
GPS-ABC	p_1	1.391	3.46e-2	1.000	0.371
(strengthened)	p_2	**0.156**	3.22e-2	1.000	
	τ	**1.422**	9.07	1.000	
GPS-ABC	p_1	**1.309**	3.47e-2	1.000	0.810
(reference)	p_2	0.241	3.05e-2	1.000	
	τ	1.799	9.01	1.000	
DNN-ABC	p_1	1.407	3.43e-2	0.938	0.143
	p_2	0.258	3.07e-2	0.979	
	τ	1.753	8.64	1.000	

Two things in this table matter, and neither favours the neural surrogate.

The two surrogates are statistically tied. Attaching Monte Carlo standard errors to every cell and comparing DNN-ABC against the strengthened GPS-ABC at the 2-MCSE level, seven of the nine parameter-by-truth comparisons are ties and the remaining two favour GPS-ABC; none

favours the network. The two are p_2 at $(1 \times 10^{-4}, 1 \times 10^{-2}, 3)$, difference $+0.050 \pm 0.039$, and τ at $(2 \times 10^{-3}, 8 \times 10^{-3}, 5)$, $+0.411 \pm 0.409$ – both marginal. We state this plainly because Study I found a consistent DNN advantage and it would be easy to assume it carries over: it does not. The reason is visible in Section 5.3 – when both surrogates fit the response surface essentially perfectly relative to its noise, the surrogate is no longer the bottleneck, and downstream accuracy is set by the identifiability of the model rather than by the quality of the emulator. A surrogate can only help where surrogate error is what limits the answer.

Every method over-covers, and the differences between them are not resolvable at this replicate count. Coverage of the nominal 95% credible interval is 0.938–1.000 for DNN-ABC and 1.000 throughout for both Gaussian processes. With sixteen replicates per cell, coverage moves in steps of $1/16 = 0.0625$, so DNN-ABC's 0.938 for p_1 is fifteen replicates out of sixteen against the GP's sixteen – a difference of one, well inside the per-cell Monte Carlo standard error (coverage MCSE ± 0.08 at this replicate count). We therefore do not claim a calibration difference between the methods. Mean interval widths are likewise close (p_1: 3.43×10^{-2} for DNN-ABC against 3.46×10^{-2} for GPS-ABC).

What the coverage column does show is that all three methods over-cover on τ, at 1.000 across the board. That is what a nearly unidentified parameter looks like rather than a defect of any surrogate: the credible interval spans most of the prior box, so it contains the truth essentially by construction. Note also that this is not a failure of the surrogate's own predictive calibration, which is near-exact on held-out data (Table 9, coverage 0.955); it is a property of the ABC acceptance kernel built on top of it.

We record one diagnostic that qualifies the comparison: DNN-ABC's acceptance rate is 0.143 against the strengthened GP's 0.371 and the reference GP's 0.810, so the three chains mix very differently at the shared proposal scale and some of the DNN's disadvantage may be tuning rather than method. The reference GP's high acceptance is itself informative – a flatter, less discriminating surrogate accepts more freely. Per-method proposal tuning is the obvious next step and is not attempted here.

Taken together, Study II is best read as a negative-to-neutral result on the surrogate question and a positive one on the modelling question. The neural surrogate matches, but does not beat, a Gaussian process on downstream inference in this regime, because the regime is one where neither is the limiting factor. What does bind is the model's intrinsic identifiability: τ carries an RMSE of roughly 1.7–1.9 time units against a prior range of 9.8, and p_1 is recovered no better than an order of magnitude by either method.

5.7 Does architecture matter here? A structural comparison

The heteroscedastic MLP of Section 5.3 treats its three inputs $(\log_{10} p_1, \log_{10} p_2, \tau)$ as an unordered feature vector. A natural question, and one a reviewer of a paper proposing a neural surrogate is entitled to ask, is whether an architecture built around a different inductive bias – a convolution, or a recurrence – does better. The simulation-based-inference literature gives a specific, testable answer before any experiment is run: Radev et al. (2020) state that a summary network's architecture "should be aligned with the probabilistic symmetry of the observed data," recommending an LSTM or a 1-D convolutional network specifically when the raw input is a time series with genuine sequential dependence, and a permutation-invariant pooling network when it is an exchangeable i.i.d. sample – with a plain feedforward network the implied default otherwise. Our three inputs

are neither: p_1, p_2 and τ are three physically distinct quantities with no temporal order and no exchangeability, so this principle predicts, *a priori*, that a convolution or a recurrence should be neutral to harmful relative to an MLP. This contrasts with settings where convolutional networks have been genuinely useful in closely related inference problems – Flagel et al. (2019) and Åkesson et al. (2021) apply CNNs to population-genetic parameter estimation, including mutation-rate-type quantities, but their raw input is a genotype-alignment *matrix* or a full stochastic-trajectory time series, data with real spatial or temporal structure for a kernel to exploit; the feedforward architecture we use throughout this paper is instead the longer-standing default in this literature for tabular/summary-statistic input (Sheehan and Song, 2016; Jiang et al., 2017). We test the prediction directly rather than assume it.

Setup. We train a 1-D CNN (two convolutional layers, kernel width 3, over the three inputs treated as adjacent positions), a GRU, and an LSTM (each unrolling the three inputs as three timesteps, reading out the final hidden state), with the same two-headed Gaussian-NLL objective, split-conformal calibration, and train/validation/test split as the deployed MLP (Section 5.5), and comparable parameter counts (2,402–3,426). Full architectural detail and code are in the repository (`Models/3D/network/model_families.py`). Each is evaluated exactly as GPS-ABC and DNN-ABC were: a surrogate-fit comparison against the irreducible noise floor of Section 5.3, and the identical full ABC-MCMC recovery task of Section 5.6 (same three truth triples, 16 replicates, 3,000 iterations, 1,000 burn-in), with GPS-ABC and DNN-ABC's numbers reused unchanged from Table 10.

Table 11: Surrogate fit quality by architecture family, test set. The irreducible noise floor is 3.451×10^{-4} (Section 5.3). The deployed FFN's row is read from the capacity study rather than restated, so it necessarily describes the same target as the rows beside it.

Architecture	Parameters	MSE (design means)	× floor	R^2	95% coverage
CNN1D (2 conv layers, k=3)	2,482	3.589×10^{-4}	1.04	0.99670	0.949
FFN 64–32 [deployed]	**2,402**	$\mathbf{3.749 \times 10^{-4}}$	**1.09**	**0.99656**	**0.952**
LSTM (hidden 24)	2,642	4.221×10^{-4}	1.22	0.99612	0.950
RNN (GRU, hidden 32)	3,426	4.520×10^{-4}	1.31	0.99585	0.949

CNN1D statistically ties the deployed FFN (4.3% lower MSE, 1.04× against 1.09× the floor – inside the seed-to-seed spread already documented in Table 6); with a kernel spanning all three positions in a single layer, a convolution on an input this short is not meaningfully more constrained than an MLP, so this is closer to a non-event than a genuine convolutional advantage. It also costs 143 µs per query against the FFN's 40, which is the wrong trade for a tie. LSTM and RNN are measurably worse (+12.6%, +20.6% MSE; both gaps an order of magnitude larger than their own across-seed standard deviation), consistent with the structural prediction: a three-step recurrence must route everything relevant about p_1 through two more hidden-state updates before it can interact with τ, a strictly harder optimization problem than an MLP's direct simultaneous access to all three inputs, for no compensating benefit on a surface with no real temporal structure to reward it.

Table 12: ABC-MCMC recovery of (p_1, p_2, τ) by architecture family, per-parameter RMSE ($\log_{10}$ scale for p_1, p_2; absolute time units for τ), averaged over the three truth triples of Table 10. Bold marks the best value in each row.

Parameter	GPS-ABC	DNN-ABC (FFN)	CNN1D-ABC	RNN-ABC	LSTM-ABC
p_1 RMSE	1.391	1.407	**1.373**	1.378	1.389
p_1 coverage	1.000	0.938	0.958	0.917	0.979
p_2 RMSE	**0.156**	0.258	0.203	0.312	0.209
p_2 coverage	1.000	0.979	0.979	0.938	0.958
τ RMSE	**1.422**	1.753	1.688	1.700	1.631
τ coverage	1.000	1.000	1.000	1.000	1.000

Attaching Monte Carlo standard errors to every one of the $3 \times 3 \times 3 = 27$ new-family, truth and parameter cells, and comparing each new family against both GPS-ABC and DNN-ABC at the 2-MCSE level (54 pairwise comparisons in total), gives a picture consistent with the fit-quality result, and a slightly starker one: 49 of 54 (91%) are statistical ties. All 5 resolved comparisons favour GPS-ABC, and *none* favours a new family anywhere. Three of the five are CNN1D-ABC and two RNN-ABC; LSTM-ABC is tied in every one of its eighteen comparisons. Swapping the feed-forward surrogate for a convolutional or recurrent one therefore changes nothing that survives its own Monte Carlo error.

Interval calibration adds nothing to the picture either way. Pooled coverage is 0.951–0.979 for the four neural families and 1.000 for both Gaussian processes, against a nominal 0.95. An earlier version of this section read a narrative into that column – the FFN under-covering, CNN1D inheriting and worsening it, LSTM correcting it – which the corrected numbers do not support: the spread across all four neural families is under three percentage points, which at sixteen replicates is at most one cell per parameter. Every method over-covers on τ at exactly 1.000, which reflects that parameter's weak identifiability rather than any property of an architecture. We therefore draw no calibration conclusion from this comparison.

Summary. Neither imposing a spatial prior (CNN1D) nor a temporal one (RNN, LSTM) on three physically unordered parameters improves point-estimation accuracy over the architecture-agnostic FFN or the Gaussian-process baseline, and two of the three impose a real cost on fit quality (LSTM +12.6%, RNN +20.6% MSE) for no offsetting benefit, while the third (CNN1D) ties the FFN at 3.6× its query cost. This is the outcome Radev et al. (2020)'s structural principle predicts, and it is why the deployed 3-D surrogate throughout this paper remains the plain heteroscedastic MLP of Section 5.3. Full per-cell results, code, and the trained checkpoints for all three architecture families are included in the repository (Section 7).

5.8 Scope and what remains open

Four limitations should be read alongside the results above.

The plating time is not the paper's. $t_p = 10$ here against Lu et al.'s $t_p = 20$, forced by the exponential cost of exact simulation. Reaching their regime requires the fast two-stage simulator; it is implemented but not yet validated against the exact simulator at that scale, so we do not report results in that regime and no direct numerical comparison with their Table 4 is claimed.

The parameter regime is not the paper's either, and the gap is wider than the plating time alone. Lu et al.'s Study II places $\log_{10} p_1, \log_{10} p_2 \in [-11, -7]$; ours span $[-5, -1.3]$. The two ranges do not overlap. The cause is the same choice of an exact simulator: at $t_p = 10$ a rate below $\approx 10^{-5}$ leaves almost every culture mutant-free. A consequence is that Study I's design rule – t_p solved so that $E[X] = 20$ at every design point, which we do follow in Section 4 – cannot be maintained here, and the expected mutant count instead ranges from 0.22 to roughly 1,100 across the design. What we report is the same model studied in a different and easier regime.

The model has no δ. The reference two-stage implementation carries no differential mutant growth rate, making this a genuine three-parameter study; Lu et al.'s full specification is four-dimensional. Our simulator threads δ through so the extension does not require a rewrite.

One convention in the reference implementation is ambiguous, and we had it wrong. A cell born at its parent's division time divides at its own later time, and the reference code contains two versions of which of those two times indexes the step function in Eq. (2) – one active, one commented out. An earlier version of this study generated its ground truth under the active convention while drawing every observed dataset under the other, so the surrogates were trained on one model and scored against data from a second. The two differ by up to 10.1% in the summary statistic, and by 2.9–7.4% at the three truth triples of Table 10.

Because the generated data records no provenance, the convention was recovered by measurement: simulating at the design points of the ground truth itself under both conventions, all sixty of the most discriminating points lie closer to the active convention (mean absolute error 0.0054 against 0.0400 in $\log_{10}$ units; paired $t = +11.3$). That is also the convention of the paper's Algorithm 3, which indexes $p(t)$ by the offspring's own accumulated lifetime. The analysis now uses it throughout, and the check is part of the test suite.

The correction is worth reporting even though it changed no conclusion. Its effect on the reported quantities was below the Monte Carlo error of the experiment – a bias of 0.013–0.031 in $\log_{10}$ units entering posteriors whose RMSE is 0.5–2.7 – because the same weak identifiability that limits this study also absorbs the misspecification. Which convention the reference *intends* remains a question for its authors; the analysis is now self-consistent under either answer, and the data-generation code can produce a paired comparison on identical design points and seeds.

6 Discussion

Across both studies, the neural surrogate's advantage over the Gaussian process is clearest exactly where the theory predicts it should be: in surrogate quality and cost scaling as the useful training-set size grows (Table 1), and this advantage compounds when the exact simulator used to build the training design is itself expensive, as it is here. In the one-dimensional case, where a Gaussian process fit on ~ 800 points is already close to a well-specified problem's optimum, the two surrogates are close to interchangeable on point accuracy and query speed, and DNN-ABC's advantage is concentrated in tighter, still-calibrated intervals. Study II sharpens the caveat considerably: there the surrogate-quality advantage vanishes entirely, because both surrogates reach the data's noise floor (Table 6), and advantages in emulator quality do not automatically translate into a better *downstream ABC posterior*: point accuracy is a near-tie, and every method over-covers its credible intervals – at 1.000 throughout for τ – despite held-out predictive calibration that a plain regression diagnostic reports as near-exact (Table 9). The over-coverage is a property of the ABC acceptance

kernel built on top of the surrogate rather than of the surrogate's own predictions. This is, we think, a broader methodological point for anyone deploying a conformally calibrated regression surrogate inside a likelihood-free sampler: split-conformal calibration (Lei et al., 2018) is a strong, distribution-free guarantee about the surrogate's own predictive interval, and Sections 4.2 and 5.5 show it holding to a high degree of precision in both studies – but it is a guarantee about the regression problem the surrogate was trained to solve, not about the coverage of whatever downstream quantity the surrogate's uncertainty is subsequently plugged into. The two can, and here do, come apart.

A second, purely architectural, finding recurs across both studies and is worth stating plainly for anyone building a similar surrogate: the single largest driver of surrogate quality in the one-dimensional architecture search (Table 1) was *not* the choice of activation function, which was a near-tie among several reasonable choices, but the presence or absence of BatchNorm (Ioffe and Szegedy, 2015). Study II qualifies even this: on that surface no architectural choice mattered at all, because every candidate already sat at the noise floor (Table 6), which is itself worth measuring before any architecture search is run. An otherwise-standard network with BatchNorm fit the one-dimensional curve eleven times worse than the identical network without it. On a smooth regression problem with a bounded, one-sided input domain, BatchNorm's per-minibatch statistics appear to be actively counter-productive; LayerNorm (Ba et al., 2016), which normalizes per-sample, avoided this failure mode entirely in the deeper three-dimensional network.

A third finding, from the architecture-family comparison of Section 5.7, is best read as a companion to this one rather than a separate result: BatchNorm was a case of an architectural component actively fighting the problem's structure, whereas CNN1D/RNN/LSTM on the two-stage inputs are a case of architectural components with *nothing to find* – there is no spatial or temporal pattern in three unordered physical parameters for a kernel or a recurrence to exploit, so the honest expectation, confirmed by Radev et al. (2020)'s stated design principle and by our own result, is indifference-to-mild-harm rather than either a large gain or a BatchNorm-scale failure. The practical lesson we would draw for anyone designing a similar surrogate is to let the symmetry of the input, not habit or a desire for architectural novelty, choose the architecture family before any search over its hyperparameters begins.

Scope and limitations. This paper is a simulation study evaluated entirely under known ground truth; we do not apply the method to a real fluctuation-experiment data set, as Lu et al. (2023) do for their own estimator in their real-data application to drug-resistance experiments. We view a real-data application, ideally to the same or a comparable fluctuation-experiment data set, as the natural and immediate next step, and we are explicit that the comparisons in this paper are not a substitute for one: a simulation study can establish that an estimator is well calibrated and competitive under a correctly specified model, but only a real-data application can speak to robustness under the model misspecification that any laboratory data set will exhibit to some degree. The one-dimensional study uses 40 replicates per cell (versus Lu et al.'s 100); Section 3.4 reports Monte Carlo standard errors throughout for exactly this reason, and Sections 4.3 and 5.6 use them to identify which point-accuracy comparisons are, and are not, resolved at this replicate count, rather than over-interpreting numerically lower MSE as a real effect in every cell. The three-dimensional study's three truth triples at a single culture count, while chosen to span the parameter space broadly, do not cover it exhaustively. The exact ABC-MCMC prior is bounded in both studies to keep the exact simulator feasible inside the sampling loop; surrogates are trained on a wider range and only queried within it. Finally, Study II's interval miscalibration is, by construction, a property of the

acceptance-kernel construction used here rather than of the surrogates' own predictive calibration, which is near-exact on held-out data; whether a kernel-side correction restores nominal coverage for both surrogates simultaneously is an open question.

7 Reproducibility

Code and data availability. All code, data, trained model weights, and the exact configuration used for every table and figure in this paper are available at `https://github.com/sletch1/DNN-ABC-mutation-rates` under a public repository, with no access restrictions. The one-dimensional ground-truth data set (Section 4) is committed directly to the repository as a plain-text, non-proprietary CSV file; the two-stage ground-truth data set (Section 5.1) is likewise committed in the same format, together with the exact simulator that generated it and the reference implementation it was ported from, so that it can be independently re-run rather than taken on faith. Every CSV's columns are self-documenting from their header row (parameter values, replicate index, and the resulting summary statistic or estimator output); no external data dictionary is required to interpret them.

Computational environment. All code is Python (minimum versions pinned in the repository's `requirements.txt`: `numpy`, `pandas`, `scipy`, `matplotlib`, `scikit-learn` for the Gaussian-process baseline, and `torch` for the neural surrogates). No GPU is required for any part of the pipeline; every model in both studies trains on CPU in seconds to minutes.

Instructions for reproduction. Each study's surrogate can be retrained, its ABC tables reproduced, and its figures regenerated with three scripted commands documented in the repository's README; the two-stage pipeline documents the same sequence – data generation, the two rounds of architecture search, the activation selection, training and conformal calibration, the ABC tables, Monte Carlo standard errors, and figure generation – in the order the results are presented in this paper.

The two-stage package also ships a validation script that re-derives, rather than asserts, three facts the analysis depends on: that the two-stage simulator reduces to the constant-rate one in both degenerate limits (checked against an independently ported constant-rate implementation, and exact on a shared seed); the size of the mutation-time convention gap reported in Section 5.8; and which convention the committed ground truth was generated under, since the data file itself records no provenance. The summary statistic is likewise recomputed from the per-culture values stored in that file rather than from a cached aggregate, so the choice of root in Section 5.1 is a single switch and cannot drift between the surrogate, the baselines and the sampler. Every number reported in Tables 3–10 and in the architecture-family comparison of Tables 11–12 (Section 5.7) is produced by these scripts directly from the committed data, with no manual post-processing; the Monte Carlo standard errors and Δ/SE columns introduced in Section 3.4 are computed from the same per-replicate output each script already writes to disk.

8 Conclusion

A heteroscedastic neural-network surrogate, calibrated by split-conformal prediction and substituted directly for the Gaussian process inside Lu et al.'s ABC-MCMC sampler, matches or improves on GPS-ABC in a direct one-dimensional reproduction of their benchmark. Extending the same construction to the two-stage model (p_1, p_2, τ), however, does *not* reproduce that advantage: there the neural and Gaussian-process surrogates are statistically tied in seven of nine comparisons with the other two favouring the Gaussian process, and we report the result rather than tune it away. The explanation we offer is a general one and is the more useful contribution of Study II. A surrogate can only improve an inference where surrogate error is what limits it; once the emulator sits at the data's irreducible noise floor – as every network we tried does, across three orders of magnitude of model size – the binding constraint becomes the identifiability of the model itself, and no better emulator can help. Measuring that floor before comparing architectures is cheap, and we would encourage it as routine practice in surrogate studies. Both the surrogate-substitution framework and the diagnosis of where its guarantees do and do not transfer downstream should apply beyond this particular mutation-rate model to other ABC procedures built on a regression surrogate for an expensive simulator.

References

Åkesson, M., Singh, P., Wrede, F., and Hellander, A. (2021). Convolutional neural networks as summary statistics for approximate Bayesian computation. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*.

Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.

Beaumont, M. A., Zhang, W., and Balding, D. J. (2002). Approximate Bayesian computation in population genetics. *Genetics*, 162(4):2025–2035.

Elfwing, S., Uchibe, E., and Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11.

Flagel, L., Brandvain, Y., and Schrider, D. R. (2019). The unreasonable effectiveness of convolutional neural networks in population genetic inference. *Molecular Biology and Evolution*, 36(2):220–238.

Hendrycks, D. and Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.

Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456.

Jiang, B., Wu, T.-Y., Zheng, C., and Wong, W. H. (2017). Learning summary statistic for approximate Bayesian computation via deep neural network. *Statistica Sinica*, 27:1595–1618.

Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., and Wasserman, L. (2018). Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111.

Lu, R., Zhu, H., and Wu, X. (2023). Estimating mutation rates in a Markov branching process using approximate Bayesian computation. *Journal of Theoretical Biology*, 565:111467.

Marin, J.-M., Pudlo, P., Robert, C. P., and Ryder, R. J. (2012). Approximate Bayesian computational methods. *Statistics and Computing*, 22(6):1167–1180.

Marjoram, P., Molitor, J., Plagnol, V., and Tavaré, S. (2003). Markov chain Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences*, 100(26):15324–15328.

Morris, T. P., White, I. R., and Crowther, M. J. (2019). Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11):2074–2102.

Radev, S. T., Mertens, U. K., Voss, A., Ardizzone, L., and Köthe, U. (2020). BayesFlow: Learning complex stochastic models with invertible neural networks. *arXiv preprint arXiv:2003.06281*.

Sheehan, S. and Song, Y. S. (2016). Deep learning for population genetic inference. *PLOS Computational Biology*, 12(3):e1004845.